FORMATION · 2 JOURS · 16 LABS

Terraform sur Microsoft Azure

Les notions théoriques avant chaque lab : comprendre d'abord, pratiquer ensuite.

Infrastructure as Code | HCL | azurerm 4.x | Terraform ≥ 1.9

Belgacem Hajji

PROGRAMME

Deux jours, un fil rouge, puis l'industrialisation

Jour 1 · Les fondamentaux

Lab	Notion
01	Installation, authentification
02	Première infrastructure
03	Outputs
04	Variables
05	Modules
06	Provisioners
07	Commandes et dérive
08	count et for_each

Jour 2 · État, équipe, réseau

Lab	Notion
09	Data sources et lifecycle
10	Lire un autre état
11	État distant Azure
12	HCP Terraform + GitHub
13	templatefile et RBAC
14	Multi-provider
15	Réseau virtuel complet
16	VM dans le VNet

LE POINT DE DÉPART

Créer une infrastructure à la main ne passe pas à l’échelle.

Des dizaines de clics dans le portail, aucune trace, impossible à refaire à l’identique, et personne ne sait ce qui a changé ni pourquoi.

INFRASTRUCTURE AS CODE

Décrire l’infrastructure dans des fichiers texte

Reproductible

Le même code produit le même résultat, en dev comme en prod.

Versionné

Git garde l’historique : qui a changé quoi, quand, pourquoi.

Relu

Une pull request permet de relire un changement avant qu’il ne parte.

Automatisable

Un pipeline CI/CD applique le code sans intervention humaine.

L'OUTIL

Terraform en une phrase

Un outil open source qui lit des fichiers .tf, calcule ce qu'il faut changer, puis appelle les API des fournisseurs cloud pour le faire.

Déclaratif

On décrit le résultat voulu, pas les étapes.

Langage HCL

Lisible par un humain, simple à apprendre.

Multi-fournisseur

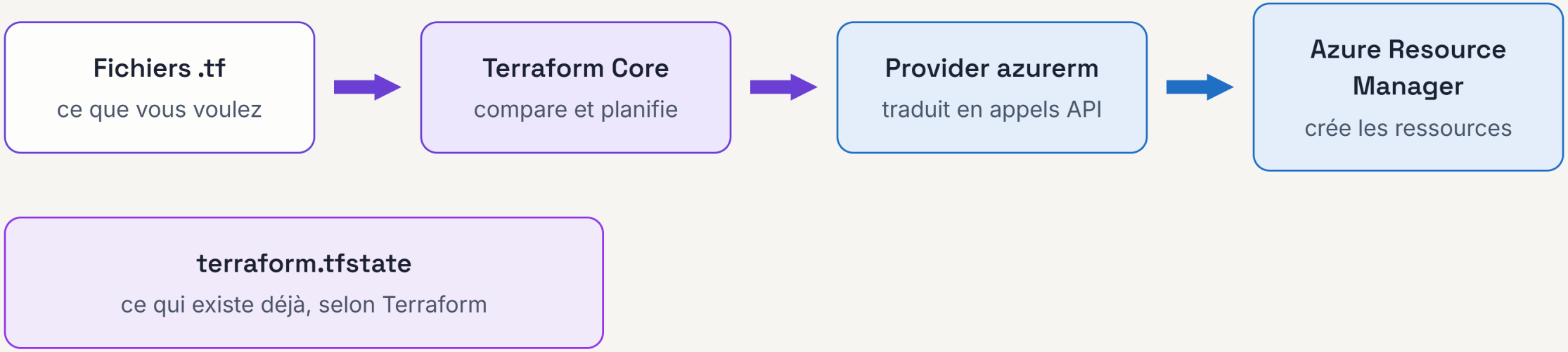
Azure, AWS, GitHub, Kubernetes… des milliers de providers.

Avec un état

Il se souvient de ce qu'il a créé.

COMMENT ÇA MARCHE

Du fichier .tf à la ressource Azure



Terraform compare trois choses : le code (le souhaité), l'état (le connu) et Azure (le réel).

LE WORKFLOW

Cinq commandes à connaître



Règle d’or

Toujours lire le plan avant de répondre « yes ». Le plan dit exactement ce qui sera créé, modifié ou détruit.

DÉCLARATIF OU IMPÉRATIF

Décrire le résultat, pas les étapes

Impératif (script)

```
az group create -n rg-web -l francecentral
az vm create -g rg-web -n vm-web ...
# relancé une 2e fois → erreur ou doublon
```

Déclaratif (Terraform)

```
resource "azurerm_resource_group" "rg" {
  name     = "rg-web"
  location = "francecentral"
}
# relancé 10 fois → "No changes"
```

LAB 01 · JOUR 1 · 30 MIN

01

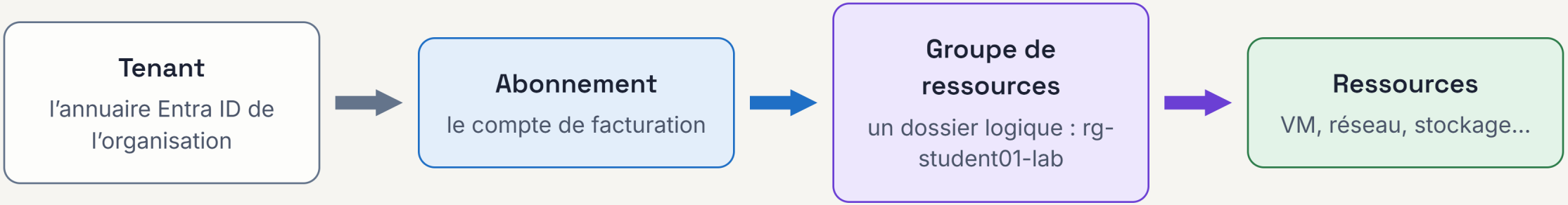
Installation et authentification

Objectif : installer les outils et prouver que Terraform parle à Azure.

Terraform CLI | Azure CLI | az login | Service Principal | variables ARM_*

LAB 01 · VOCABULAIRE AZURE

Où vivent les ressources Azure



LAB 01 · AUTHENTIFICATION

Deux façons de se connecter

	az login	Service Principal
Qui ?	Vous, votre compte	Une identité applicative dédiée
Comment ?	Navigateur, MFA	client_id + secret, sans humain
Droits	Tous les vôtres	Seulement ceux attribués
CI/CD	Impossible	Fait pour ça

LAB 01 · VARIABLES ARM_*

Le provider lit ses identifiants dans l'environnement

```
# ~/.azure-tf.env  (chmod 600, jamais dans Git)
export ARM_CLIENT_ID="<appId>"
export ARM_CLIENT_SECRET="<secret>"
export ARM_TENANT_ID="<tenant>"
export ARM_SUBSCRIPTION_ID="<abonnement>"

provider "azurerm" {
  features {}   # obligatoire, même vide
}
```

Aucun secret dans le code

Le provider lit ARM_* tout seul : rien à écrire dans les fichiers .tf.

subscription_id

Obligatoire depuis azurerm 4, fourni par ARM_SUBSCRIPTION_ID.

LAB 02 · JOUR 1 · 60 MIN

Première infrastructure Azure

02

Objectif : déployer une VM Linux avec tout son réseau, et savoir lire un plan.

bloc resource · références · dépendances · terraform plan · + ~ -/+ · state

LAB 02 · LE LANGAGE HCL

Anatomie d’une ressource

```
resource "azurerm_resource_group" "rg" {
  name     = "rg-student01-lab"
  location = "francecentral"
}
```

Le type

azurerm_resource_group : provider + sorte de ressource.

Le nom local

rg : le nom que l’on utilise dans le code, pas dans Azure.

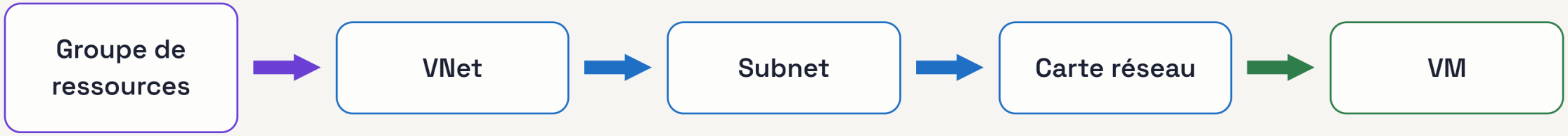
Les arguments

name, location : la configuration voulue.

LAB 02 · RÉFÉRENCES

Une référence crée une dépendance

```
resource "azurerm_subnet" "web" {
  virtual_network_name = azurerm_virtual_network.vnet.name   # ← référence
}
```



Terraform lit les références, construit un graphe et crée tout dans le bon ordre, en parallèle quand c'est possible.

LAB 02 · LIRE UN PLAN

Quatre symboles à reconnaître

+ création

Nouvelle ressource dans le code.

~ mise à jour

Modification sur place : un tag, une taille de VM.

-/+ recréation

L'attribut ne peut pas changer : détruire puis recréer. Repérez « forces replacement ».

- destruction

Ressource retirée du code.

LAB 02 · L'ÉTAT

terraform.tfstate, la mémoire de Terraform

Ce qu'il contient

La liste de tout ce que Terraform a créé, avec les identifiants Azure et tous les attributs.

Pourquoi c'est vital

Sans lui, Terraform ne sait plus que la VM existe déjà et voudrait la recréer.

Les règles

Ne jamais le modifier à la main. Ne jamais le versionner : il contient des secrets.

LAB 03 · JOUR 1 · 20 MIN

Les outputs

03

Objectif : exposer les informations utiles après un déploiement : IP, DNS, commande SSH.

output | terraform output | -raw / -json | interpolation | sensitive

LAB 03 · OUTPUTS

Afficher ce qui est utile après l’apply

```
output "public_ip" {
  value = azurerm_public_ip.web.ip_address
}

output "ssh_command" {
  value = "ssh azureuser@${azurerm_public_ip.web.ip_address}"
}
```

À quoi ça sert

Lire une IP, alimenter un script, passer une valeur à un autre projet.

sensitive = true

Masque la valeur à l’écran, mais elle reste en clair dans l’état.

LAB 04 · JOUR 1 · 50 MIN

04

Les variables

Objectif : rendre le code réutilisable : plus aucune valeur propre au participant en dur.

variable · types · validation · terraform.tfvars · TF_VAR_* · lookup

LAB 04 · DÉCLARER

Une variable = un paramètre du projet

```hcl
variable "identity" {
  description = "Identifiant du participant"
  type        = string

  validation {
    condition     = can(regex("^[a-z0-9]{3,12}$", var.identity))
    error_message = "3 à 12 minuscules ou chiffres."
  }
}

name = "rg-${var.identity}-lab"   # utilisation
```

Types

string, number, bool, list, map, object.

Sans default

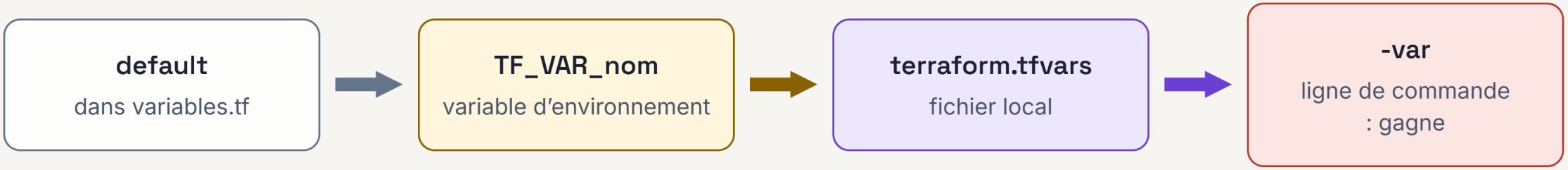
La variable est obligatoire.

validation

Refuse une mauvaise valeur dès le plan.

LAB 04 · FOURNIR LES VALEURS

Quatre sources, une priorité



LAB 04 · LE PIÈGE CLASSIQUE

ARM_* et TF_VAR_* ne servent pas à la même chose

ARM_*

Lues par le PROVIDER azurerm pour s'authentifier. Elles ne remplissent aucune variable Terraform.

TF_VAR_nom

Lues par TERRAFORM pour remplir variable « nom ». TF_VAR_identity remplit var.identity.

Entre guillemets

« ARM_SUBSCRIPTION_ID » est une simple chaîne : HCL ne lit jamais l'environnement.

LAB 05 · JOUR 1 · 60 MIN

Les modules

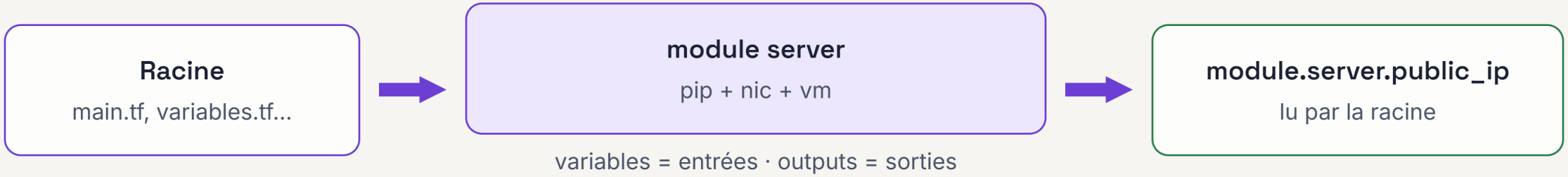
05

Objectif : ranger le serveur dans un module réutilisable, sans rien recréer.

module · entrées / sorties · moved · Terraform Registry · tls_private_key

LAB 05 · LE CONCEPT

Un module est une boîte avec des entrées et des sorties



variables = entrées · outputs = sorties

LAB 05 · APPELER UN MODULE

Passer les valeurs, récupérer les résultats

```
module "server" {
  source = "./modules/server"

  identity  = var.identity              # entrée
  subnet_id = azurerm_subnet.web.id     # entrée
}

output "public_ip" {
  value = module.server.public_ip       # sortie
}
```

source

Un dossier local ou un module du registre public.

terraform init

Obligatoire après l'ajout d'un module.

LAB 05 · REFACTORISER

moved : déplacer sans détruire

Sans moved, une nouvelle adresse = une nouvelle ressource :

```
azurerm_public_ip.web
  → module.server.azurerm_public_ip.web

Plan: 3 to add, 0 to change, 3 to destroy
```

Avec moved, Terraform renomme dans l'état :

```
moved {
  from = azurerm_public_ip.web
  to   = module.server.azurerm_public_ip.web
}
# Plan: 0 to add, 0 to change, 0 to destroy
```

LAB 05 · RÉUTILISER

Le registre public et le provider tls

Terraform Registry

registry.terraform.io : des milliers de modules prêts à l'emploi, dont Azure/naming et les Azure Verified Modules.

Toujours épingler

version = « ~> 0.4 » : on choisit quand passer à une nouvelle version.

Provider tls

Génère une paire de clés SSH. Attention : la clé privée est stockée en clair dans l'état.

LAB 06 · JOUR 1 · 45 MIN

Provisioners et cloud-init

06

Objectif : installer un serveur web sur la VM juste après sa création.

connection · provisioner file · remote-exec · self · custom_data

LAB 06 · PROVISIONERS

Exécuter des commandes sur la VM créée

```
connection {
  type        = "ssh"
  user        = "azureuser"
  private_key = var.private_key
  host        = self.public_ip_address
}

provisioner "remote-exec" {
  inline = ["sudo apt-get install -y apache2"]
}
```

self

Désigne la ressource en cours de création.

Seulement à la création

Ajouter un provisioner ne fait rien. Pour le rejouer : apply -replace.

En cas d’échec

La ressource est marquée tainted et sera recréée.

LAB 06 · L'ALTERNATIVE RECOMMANDÉE

Provisioners ou cloud-init ?

	Provisioners	custom_data (cloud-init)
Connexion SSH depuis Terraform	Obligatoire	Aucune
Port 22 ouvert pour l'outil	Oui	Non
Erreurs visibles	Dans Terraform	Dans la VM (logs cloud-init)
Modifier le script	Rien ne se passe	La VM est recréée
Recommandé en production	En dernier recours	Oui

LAB 07 · JOUR 1 · 60 MIN

Commandes, dérive et plan enregistré

07

Objectif : inspecter son projet, détecter une modification faite à la main et y réagir.

console · graph · state · drift · -refresh-only · plan -out · fmt / validate

LAB 07 · BOÎTE À OUTILS

Les commandes d’inspection

Commande	À quoi elle sert
terraform console	Tester une expression ou une fonction en direct
terraform graph	Dessiner le graphe des dépendances
terraform state list / show	Lister et détailler ce que contient l’état
terraform output	Relire les sorties
terraform fmt	Mettre le code au format standard
terraform validate	Vérifier syntaxe et références, sans Azure

LAB 07 · LA DÉRIVE

Quand la réalité ne correspond plus au code

Le code
ce que l'on veut

L'état
ce que Terraform croit

Azure
ce qui existe vraiment

LAB 07 · RÉAGIR À UNE DÉRIVE

Trois décisions possibles

Restaurer

La modification est une erreur : terraform apply remet ce que dit le code.

Adopter

La modification est légitime : on l'écrit dans le code.

Ignorer

L'attribut est géré ailleurs : lifecycle ignore_changes (lab 09).

```
terraform plan -refresh-only    # montre UNIQUEMENT la dérive
terraform apply -refresh-only   # accepte la dérive dans l'état, sans toucher Azure
```

LAB 07 · PLAN ENREGISTRÉ

Appliquer exactement ce qui a été relu

```
terraform plan -out=tfplan
terraform show tfplan     # relecture
terraform apply tfplan    # sans nouvelle question

# si l'état a changé entre-temps :
Error: Saved plan is stale
```

En équipe et en CI/CD

On relit et on approuve un plan, puis on applique ce plan précis.

Protection

Un plan devenu obsolète est refusé.

LAB 08 · JOUR 1 · 60 MIN

Les méta-arguments count et for_each

08

Objectif : créer plusieurs serveurs identiques, et comprendre pourquoi for_each est plus sûr.

count | count.index | splat [*] | for_each | each.key | dynamic

LAB 08 · COUNT

N copies d’une ressource, numérotées

```
resource "azurerm_linux_virtual_machine" "web" {
  count = 2
  name  = "vm-web-${count.index + 1}"
}

# adresses : web[0], web[1]
output "ips" {
  value = azurerm_public_ip.web[*].ip_address
}
```

count.index

Commence à 0, d’où le + 1 dans le nom.

Le splat [*]

Récupère un attribut de toutes les copies.

LAB 08 · LE PIÈGE

Retirer un élément au milieu décale tout

[0] web-1	[1] web-2	[2] web-3

On veut supprimer seulement web-1… mais count ne connaît que des positions :

[0] devient web-2 recréée	[1] devient web-3 recréée	[2] détruite

LAB 08 · FOR_EACH

Des clés stables plutôt que des positions

```
resource "azurerm_linux_virtual_machine" "web" {
  for_each = toset(["web-1", "web-2"])
  name     = "vm-${each.key}"
}

# adresses : web["web-1"], web["web-2"]
# retirer "web-1" ne touche pas "web-2"
```

count

Copies interchangeables, ou ressource optionnelle : count = var.x ? 1 : 0.

for_each

Éléments qui ont une identité propre : serveurs, sous-réseaux, règles.

LAB 09 · JOUR 2 · 45 MIN

Sources de données et cycle de vie

09

Objectif : lire ce qui existe déjà dans Azure et contrôler quand Terraform recrée ou détruit.

data azurerm_platform_image client_config ignore_changes prevent_destroy

LAB 09 · DATA

resource crée, data lit

```hcl
data "azurerm_platform_image" "ubuntu" {
  location  = "francecentral"
  publisher = "Canonical"
  offer     = "ubuntu-24_04-lts"
  sku       = "server"
}

version = data.azurerm_platform_image.ubuntu.version
```

resource

Terraform crée, modifie et détruit l'objet.

data

Terraform lit un objet existant à chaque plan, sans jamais le modifier.

Attention

Une valeur lue peut changer : une nouvelle image Ubuntu, et les VM seraient recréées.

LAB 09 · LIFECYCLE

Quatre options pour contrôler le cycle de vie

ignore_changes

Ignorer un attribut modifié ailleurs : image, tags posés par une policy.

prevent_destroy

Refuser tout plan qui détruirait la ressource : base de données, coffre.

create_before_destroy

Créer la nouvelle avant de supprimer l'ancienne : moins de coupure.

replace_triggered_by

Recréer quand une autre ressource change : un script de configuration.

LAB 10 · JOUR 2 · 20 MIN

Lire l’état d’un autre projet

10

Objectif : faire communiquer deux projets Terraform séparés.

terraform_remote_state

outputs = contrat

architecture en couches

LAB 10 · PROJETS EN COUCHES

Un projet lit les outputs d’un autre



Seulement les outputs

Les ressources internes restent invisibles : les outputs sont le contrat entre équipes.

Accès sensible

Lire un état, c’est pouvoir lire tout son contenu, secrets compris.

LAB 11 · JOUR 2 · 45 MIN

11

État distant dans Azure Storage

Objectif : partager et protéger l'état, et empêcher deux apply en même temps.

backend azurerm | Storage Account | verrou (lease) | -backend-config | -migrate-state

LAB 11 · LE PROBLÈME

L’état local ne convient pas au travail en équipe

Pas de partage

Chacun a sa copie sur son poste : laquelle est la bonne ?

Pas de verrou

Deux apply simultanés peuvent corrompre l’état.

Risque de perte

Un disque perdu, un rm malheureux : Terraform oublie tout.

Secrets sur les postes

L’état contient des valeurs sensibles en clair.

LAB 11 · LA SOLUTION

Le backend azurerm

```
terraform {
  backend "azurerm" {
    container_name = "tfstate"
    key            = "fil-rouge.tfstate"
  }
}

terraform init \
  -backend-config="storage_account_name=$TFSTATE_SA"
```

Verrou

Pendant un apply, le blob est loué (lease) : un 2e apply attend ou échoue.

Pas de variables

Le backend est lu avant les variables : valeurs passées à l'init.

Versioning

Le stockage garde les versions précédentes de l'état.

LAB 12 · JOUR 2 · 75 MIN

12

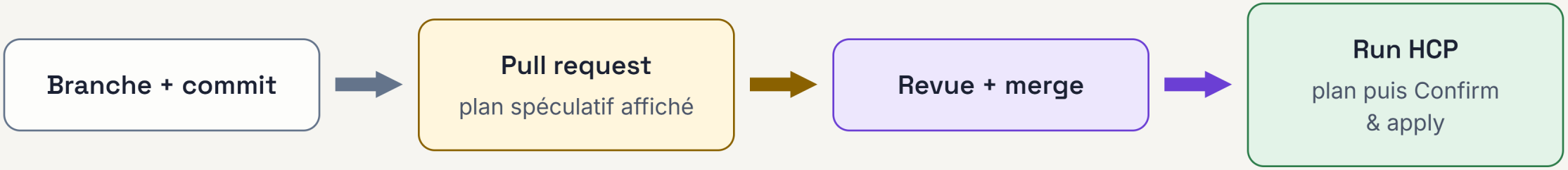
HCP Terraform et GitHub

Objectif : travailler comme une équipe : chaque changement passe par une pull request.

workspace | workflow VCS | plan spéculatif | variables sensibles | runners

LAB 12 · LE WORKFLOW GITOPS

Du commit à l'infrastructure



LAB 12 · VARIABLES DU WORKSPACE

Deux catégories de variables

Terraform

Remplissent les blocs variable : identity, allowed_ssh_cidr. Équivalent d'un terraform.tfvars.

Environment

Variables d'environnement du runner : ARM_CLIENT_ID, ARM_CLIENT_SECRET (Sensitive).

Runner éphémère

Le serveur disparaît après chaque run : aucun fichier local ne survit.

LAB 13 · JOUR 2 · 40 MIN

templatefile et Azure RBAC

13

Objectif : générer un fichier à partir d'un modèle et donner des droits précis sur un stockage.

templatefile | jsondecode | rôle personnalisé | attribution de rôle | Actions / DataActions

LAB 13 · TEMPLATEFILE

Un modèle + des valeurs = un fichier

templates/role.json.tpl

```
{
  "Name": "${role_name}",
  "AssignableScopes": ["${scope}"]
}
```



main.tf

```
templatefile("templates/role.json.tpl", {
  role_name = "role-student01-storage"
  scope     = data.azurerm_resource_group.rg.id
})
```

LAB 13 · AZURE RBAC

Qui peut faire quoi, et où

Qui ?
une identité : utilisateur, SP

Quoi ?
un rôle : liste d'actions autorisées

Où ?
une portée : abonnement, groupe, ressource

LAB 14 · JOUR 2 · 45 MIN

14

Multi-provider : Azure et GitHub

Objectif : utiliser les données d’un provider dans les ressources d’un autre.

plusieurs providers | GITHUB_TOKEN | github_ip_ranges | règles NSG sortantes | depends_on

LAB 14 · MULTI-PROVIDER

Un projet, plusieurs fournisseurs

```
required_providers {
  azurerm = { source = "hashicorp/azurerm" }
  github  = { source = "integrations/github" }
}

data "github_ip_ranges" "gh" {}

destination_address_prefixes = data.github_ip_ranges.gh.pages_ipv4
```

Identifiants

Chaque provider a les siens : ARM_* pour Azure, GITHUB_TOKEN pour GitHub.

depends_on

Seulement quand aucune référence ne relie deux ressources.

LAB 14 · FILTRER LA SORTIE

Les règles NSG sont lues par priorité

Priorité	Règle	Effet
100	Allow vers GitHub Pages	Autorisé
4000	Deny vers Internet	Tout le reste bloqué
65001	AllowInternetOutBound (défaut)	Jamais atteinte

LAB 15 · JOUR 2 · 50 MIN

Réseau virtuel complet

15

Objectif : construire un VNet avec sous-réseaux publics et privés, comme en entreprise.

VNet | sous-réseaux | for ... if | route table | NAT Gateway

Les mêmes idées, d’autres objets

AWS	Azure
VPC	Virtual Network (VNet)
Subnet lié à une zone	Subnet régional, la zone se choisit sur la VM
Internet Gateway	Pas de ressource : routes système
NAT Gateway	NAT Gateway
Security Group	Network Security Group (NSG)

LAB 15 · ARCHITECTURE

Sous-réseaux publics et privés

Sous-réseaux publics

10.0.1–3.0/24

VM avec IP publique : entrée et sortie Internet

Sous-réseaux privés

10.0.4–6.0/24

Pas d'IP publique : sortie seulement, par la NAT Gateway

```
public_subnets = {
  for k, v in local.subnets :
  k ⇒ v if v.public
}
```

LAB 16 · JOUR 2 · 40 MIN

VM dans le VNet

16

Objectif : placer une VM dans le bon sous-réseau et vérifier son réseau de l’intérieur.

zone de disponibilité | NSG sur la carte réseau | IMDS | rebond SSH

LAB 16 · À OBSERVER

Ce que la VM voit de son réseau

La zone

Fixée sur la VM et son IP publique, pas sur le sous-réseau.

NSG sur la NIC

Protège une seule VM ; sur le subnet, il protège toutes les VM.

10.0.1.1

La passerelle virtuelle du sous-réseau.

169.254.169.254

IMDS : la VM interroge ses propres métadonnées.

POUR CONCLURE

Les réflexes à garder

1. Lire le plan avant chaque apply, surtout les -/+.
2. Aucun secret dans le code, les tfvars ou Git.
3. Variables typées et validées.
4. Modules et versions épinglées.
5. moved avant de renommer.
6. for_each plutôt que count.
7. État distant, verrouillé, versionné.
8. Changements par pull request.
9. fmt et validate avant chaque commit.
10. terraform destroy en fin de journée.

FIN DE LA FORMATION

Merci ! Des questions ?

Le nettoyage avant de partir : terraform destroy dans chaque lab, puis scripts/nettoyage.sh pour vérifier qu'il ne reste rien.